Heart Disease Risk Prediction

An end-to-end MLOps pipeline: experiment tracking, CI/CD, Docker, Kubernetes and monitoring

Course	Machine Learning Operations (MLOps) - AIMLCZG523, Assignment 01
Student name	[Your full name]
Student ID	[Your BITS ID]
Code repository	https://github.com/Subham7225/heart-disease-mlops
Submission date	20 October 2026

Headline result: a tuned logistic-regression pipeline reaches ROC-AUC 0.968 and accuracy 0.885 on a stratified 20% hold-out set (5-fold CV ROC-AUC 0.904 ± 0.014). It is served by a FastAPI container running as two replicas behind a Kubernetes LoadBalancer, and monitored with Prometheus and Grafana.

Contents

1. Problem statement and approach.....
   1.1 Architecture.....
2. Setup and installation.....
3. Data acquisition and EDA.....
   3.1 Data quality.....
   3.2 Distributions and relationships.....
4. Feature engineering and model development.....
   4.1 Preprocessing pipeline.....
   4.2 Models, tuning and selection.....
5. Experiment tracking with MLflow.....
6. Model packaging and reproducibility.....
7. CI/CD pipeline and automated testing.....
8. Model containerisation.....
9. Production deployment on Kubernetes.....
10. Monitoring and logging.....
11. Access instructions and deliverables.....
   11.1 Limitations and next steps.....

1. Problem statement and approach

The goal is to predict whether a patient has heart disease from 13 routinely collected clinical measurements, and to ship that model as a reproducible, tested, containerised and monitored API. The work follows the nine assignment tasks, but every stage is wired together so that one command sequence goes from raw data to a running, observable service.

Design principles used throughout:

- **One preprocessing object.** Imputation, scaling and encoding live inside a single scikit-learn Pipeline, saved together with the model, so training and serving can never drift apart.
- **Configuration over code.** Split ratio, seed, CV folds and the full hyper-parameter grids are in configs/default.yaml.
- **Self-contained image.** The Docker build downloads the data and trains the model, so any machine with Docker can reproduce the exact served artefact.
- **Fail fast.** CI runs lint, training and tests before the Docker job is allowed to start; any non-zero exit stops the pipeline with a clear log.

1.1 Architecture

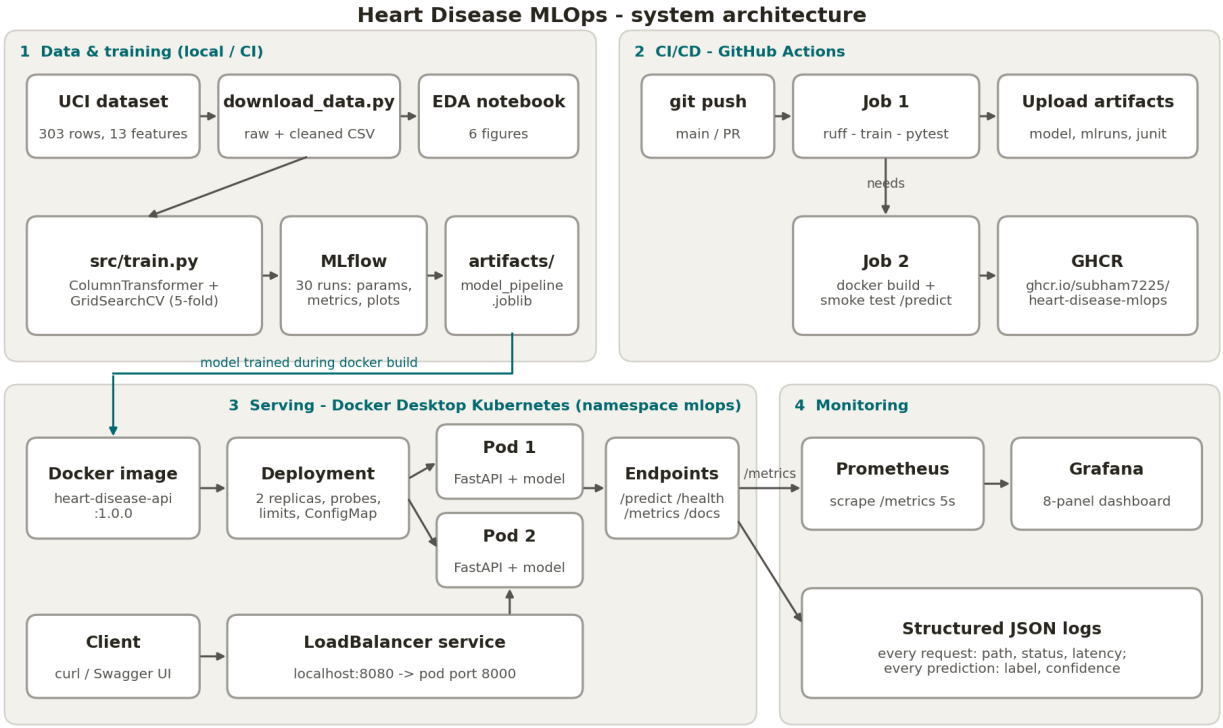


Figure 1. System architecture: data and training, CI/CD, Kubernetes serving and monitoring.

2. Setup and installation

Requirements: Python 3.11+ (tested on 3.13.5 on macOS and 3.11 in CI and Docker), Docker Desktop 4.x with Kubernetes enabled, and git. All Python dependencies are pinned in requirements.txt.

```
git clone https://github.com/Subham7225/heart-disease-mlops.git && cd heart-disease-mlops
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python data/download_data.py                # raw + cleaned CSV
python -m src.train --config configs/default.yaml  # tuning, MLflow, saves artifacts/
ruff check . && pytest -v                   # lint + 11 unit tests
mlflow ui --backend-store-uri ./mlruns --port 5001 # experiment browser
docker build -t heart-disease-api:1.0.0 . && docker run -p 8000:8000 heart-disease-api:1.0.0
```

```
./deployment/deploy_local.sh                  # Kubernetes deploy + endpoint checks
docker compose -f monitoring/docker-compose.yml up -d --build  # Prometheus + Grafana
```

Repository layout:

Path	Contents
data/	download_data.py, raw/heart.csv, processed/heart_clean.csv (cleaned dataset)
notebooks/01_eda.ipynb	Executed EDA notebook; figures exported to reports/figures/
src/	preprocess.py, train.py, evaluate.py, inference.py, config.py
app/main.py	FastAPI service: /predict, /health, /metrics, /docs
scripts/predict_sample.py	Offline inference with the saved pipeline
tests/	Pytest suite for preprocessing, model training and the API
.github/workflows/ci.yml	GitHub Actions: lint, train, test, artifacts, Docker build, smoke test, GHCR push
deployment/	Kubernetes namespace, ConfigMap, Deployment, LoadBalancer Service, Ingress, deploy script
monitoring/	Prometheus config, Grafana provisioning and dashboard, docker-compose stack
artifacts/	model_pipeline.joblib, metrics.json, evaluation plots, CV results
screenshots/	Evidence used in this report

3. Data acquisition and EDA

The Cleveland subset of the UCI Heart Disease dataset (303 patients, 13 features) is downloaded by data/download_data.py. The original 0-4 severity label num is converted to a binary target (1 = any disease). The script writes the raw file, with missing values kept for the pipeline to impute, and a cleaned copy with imputed values, which is committed as the cleaned dataset deliverable.

3.1 Data quality

- No duplicate rows. Only ca (4 rows) and thal (2 rows) have missing values, together under 2% of rows, so no rows are dropped and both are imputed with the most frequent value.
- Classes are close to balanced: 164 without disease (54%) and 139 with disease (46%). Splits are still stratified, and precision, recall and ROC-AUC are reported alongside accuracy.

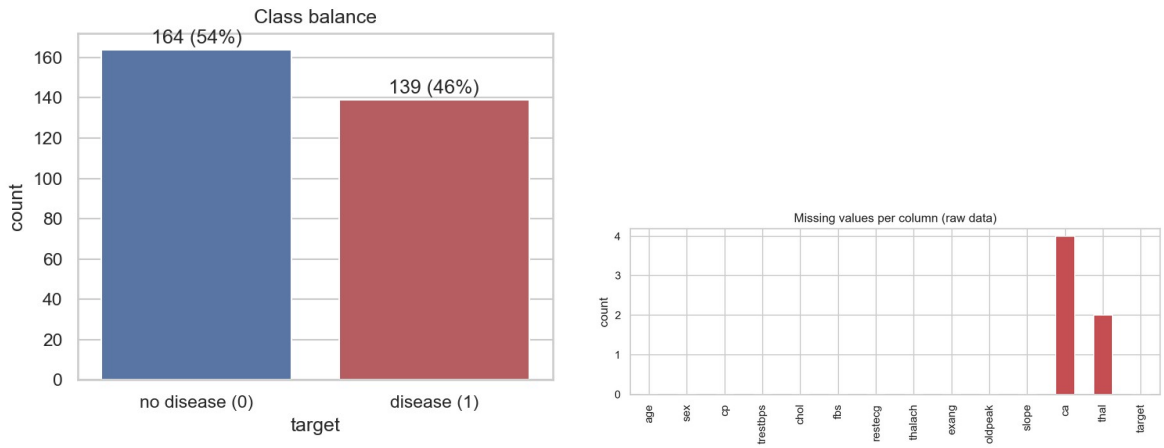


Figure 2. Class balance (left) and missing values per column in the raw data (right).

3.2 Distributions and relationships

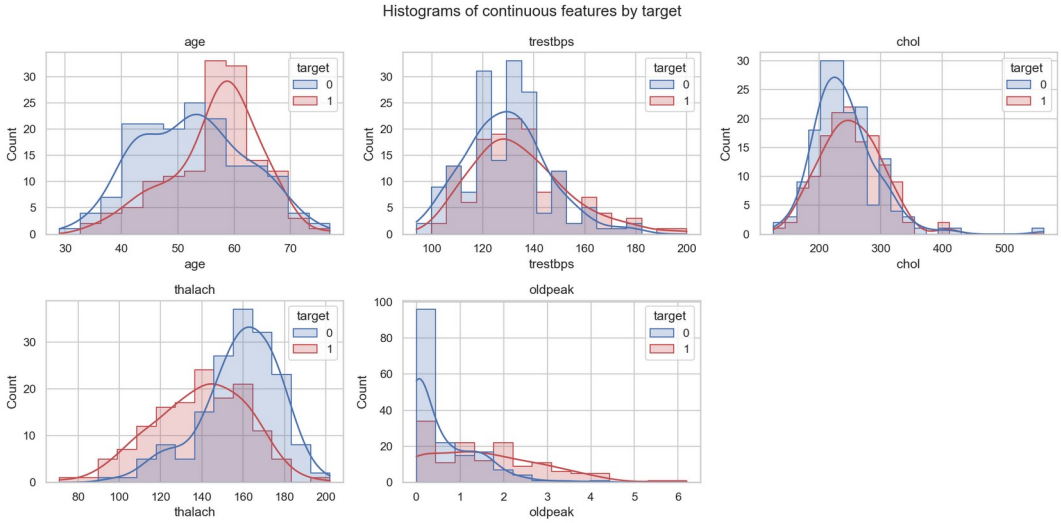


Figure 3. Histograms of the continuous features, split by target.

Patients with disease have a clearly lower maximum heart rate (thalach) and higher ST depression (oldpeak), and they skew older. Cholesterol and resting blood pressure overlap heavily between classes and contain a few high outliers (cholesterol up to 564 mg/dl). These are plausible clinical values, so they are kept and scaled rather than removed.

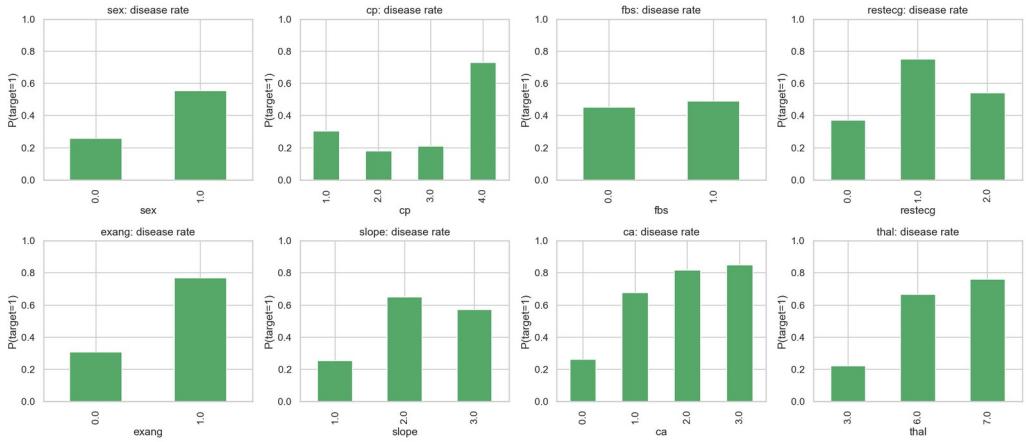


Figure 4. Disease rate for each level of the categorical features.

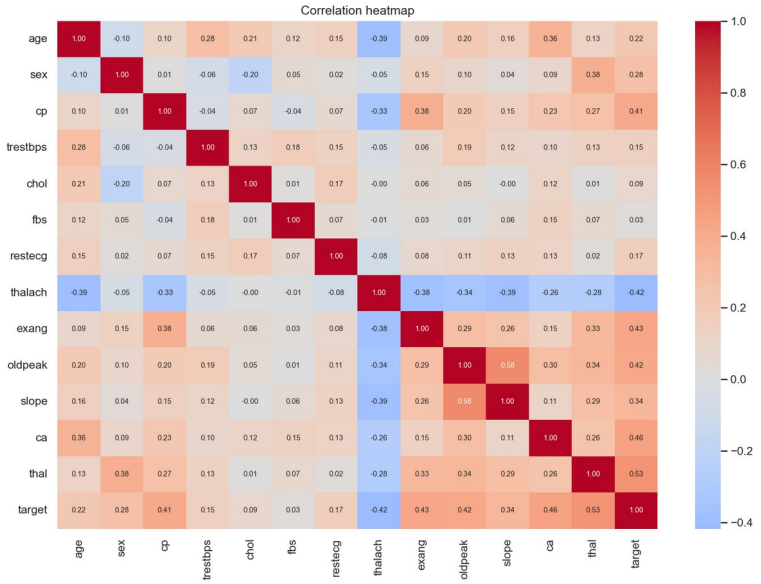


Figure 5. Pearson correlation heatmap including the target.

The strongest linear associations with the target are thal (0.52), ca (0.46), exang (0.43), oldpeak (0.42), thalach (-0.42) and cp (0.41). Chest-pain type, thal and slope are nominal codes whose disease rates do not change monotonically with the code value, which is why they are one-hot encoded rather than treated as numbers.

4. Feature engineering and model development

4.1 Preprocessing pipeline

Group	Columns	Transformations
Continuous	age, trestbps, chol, thalach, oldpeak	Median imputation, StandardScaler
Binary	sex, fbs, exang	Most-frequent imputation (kept as 0/1)
Categorical	cp, restecg, slope, ca, thal	Most-frequent imputation, OneHotEncoder(handle_unknown="ignore")

The ColumnTransformer expands the 13 inputs into 25 model features. The ignore setting means an unexpected category at prediction time produces a zero vector instead of an error.

4.2 Models, tuning and selection

Two model families were trained on the same stratified 80/20 split (seed 42), with 242 training rows and 61 test rows. Hyper-parameters were tuned with GridSearchCV using stratified 5-fold cross-validation, refitting on ROC-AUC because ranking risk matters more than a single threshold. The final model was chosen by ROC-AUC on the untouched hold-out set.

Model	Grid searched	Candidates	Best parameters
Logistic Regression	C in {0.01, 0.1, 1, 10}; L2; lbfgs	4	C = 1
Random Forest	n_estimators {200, 400}, max_depth {4, 6, 8}, min_samples_split {2, 4}, min_samples_leaf {1, 2}	24	n_estimators = 400, max_depth = 4, min_split = 2, min_leaf = 1

Tuning notes: in logistic regression, C = 0.01 under-fits (CV ROC-AUC 0.886, accuracy 0.785). Performance plateaus from C = 0.1 to 10 (ROC-AUC about 0.902-0.904). For the random forest, shallow trees (max_depth = 4) generalised best, which is consistent with the small dataset.

Metric	LogReg CV (mean ± sd)	LogReg hold-out	RF CV (mean ± sd)	RF hold-out
Accuracy	0.847 ± 0.010	0.885	0.834 ± 0.036	0.885
Precision	0.877 ± 0.054	0.839	0.853 ± 0.064	0.839
Recall	0.783 ± 0.046	0.929	0.783 ± 0.089	0.929
F1	0.825 ± 0.007	0.881	0.811 ± 0.044	0.881
ROC-AUC	0.904 ± 0.014	0.968	0.897 ± 0.022	0.950

Selected model: logistic regression. Both models make identical hold-out predictions at the 0.5 threshold (accuracy 0.885, recall 0.929). Logistic regression ranks patients better (ROC-AUC 0.968 vs 0.950), varies less across folds (ROC-AUC sd 0.014 vs 0.022) and is directly interpretable through its coefficients. High recall matters here, because a missed disease case is costlier than a false alarm.

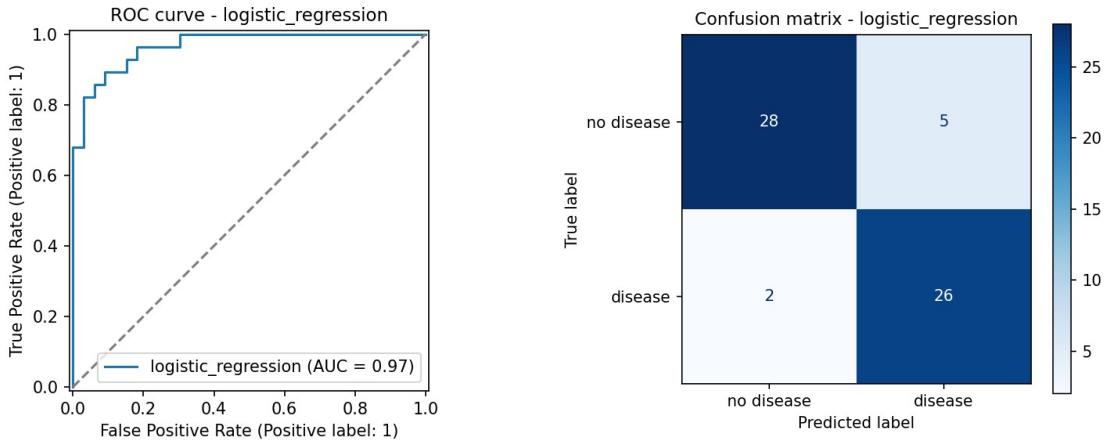


Figure 6. Hold-out ROC curve and confusion matrix of the selected logistic-regression pipeline.

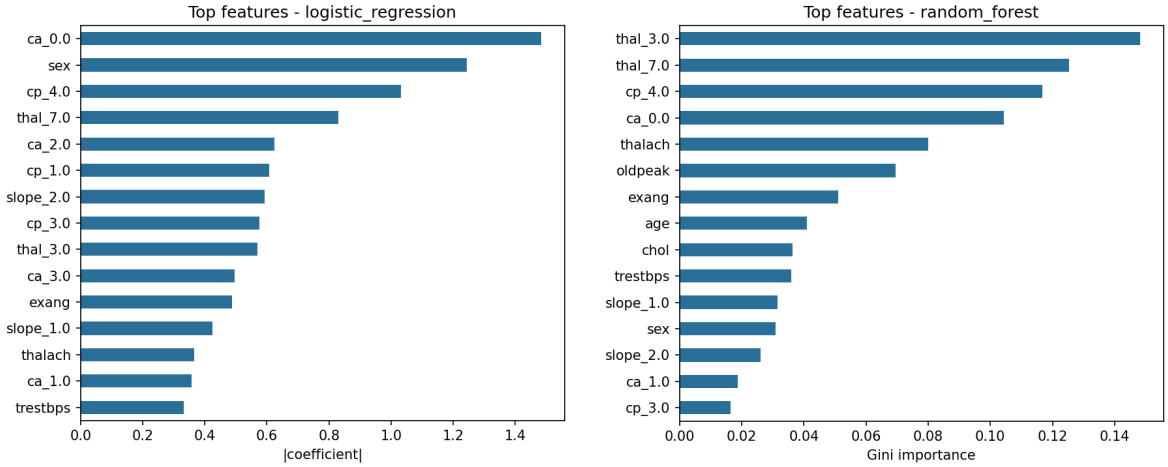


Figure 7. Top features: absolute logistic-regression coefficients (left) and random-forest Gini importance (right).

5. Experiment tracking with MLflow

src/train.py logs to a local MLflow file store (experiment heart-disease-classification). Each model family gets one parent run, and every grid candidate is a nested child run, giving 30 runs in total (2 parents, 4 + 24 candidates). For each run MLflow records:

- **Parameters:** the candidate hyper-parameters, plus split sizes, CV folds and the number of candidates on the parent run.
- **Metrics:** cv_* for every candidate; cv_mean_*, cv_std_* and test_* (accuracy, precision, recall, F1, ROC-AUC) for the best candidate.
- **Artifacts:** ROC curve, confusion matrix, feature-importance plot, the complete GridSearch CV table (CSV) and the fitted sklearn pipeline logged as an MLflow model with an input example and signature.

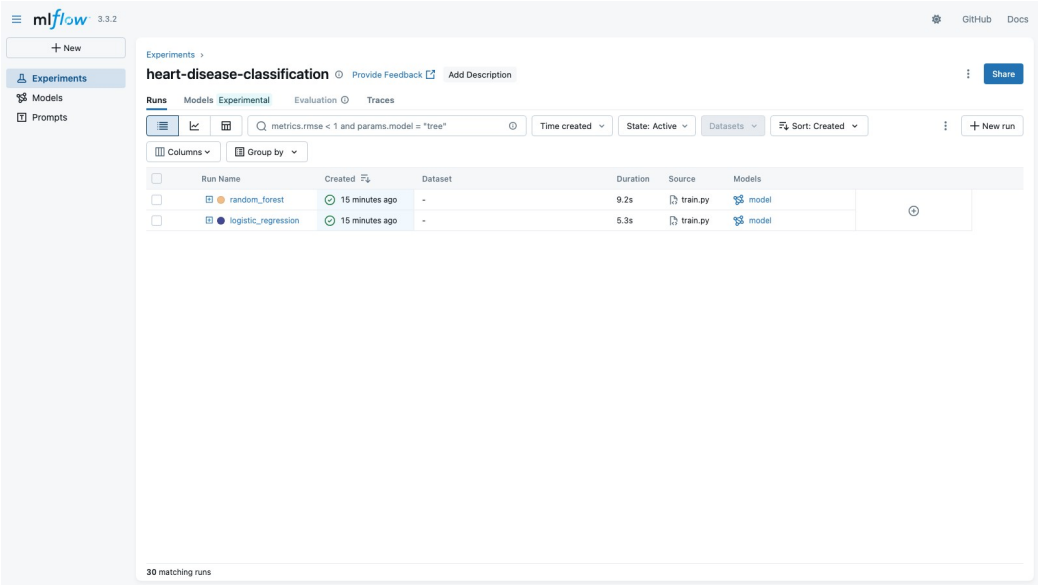


Figure 8. MLflow experiment view with both model-family runs (candidate runs nested underneath).

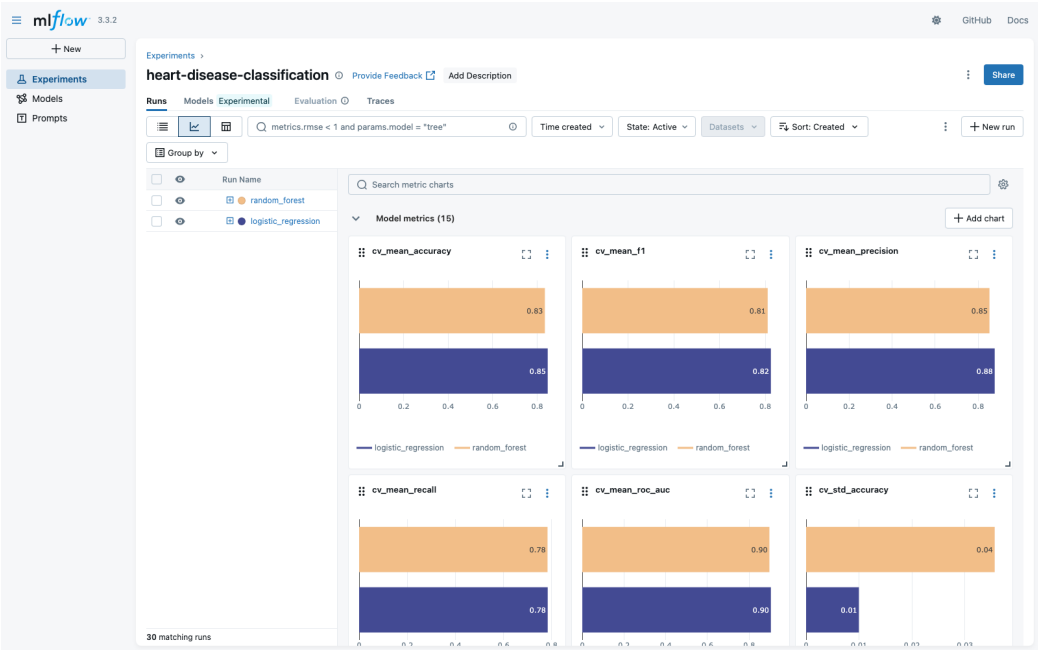


Figure 9. MLflow chart view comparing cross-validated metrics of the two tuned models.

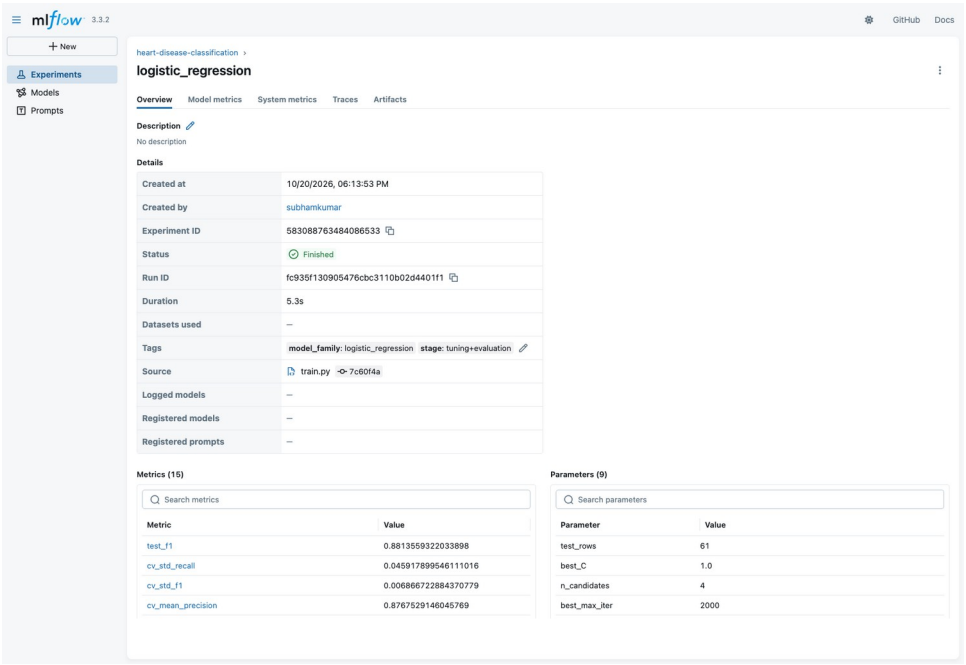


Figure 10. Logistic-regression run: logged parameters, metrics and tags.

6. Model packaging and reproducibility

- **Model format:** artifacts/model_pipeline.joblib holds the full preprocessing + model pipeline. The same pipeline is also logged in MLflow format (MLmodel, conda.yaml, requirements.txt), so it can be loaded with mlflow.sklearn.load_model.
- **Metadata:** artifacts/metrics.json stores the best parameters, CV and hold-out metrics and the MLflow run IDs. label_mapping.json maps 0/1 to low_risk/high_risk.
- **Environment:** requirements.txt pins every direct dependency, e.g. scikit-learn 1.7.1, pandas 2.3.2, mlflow 3.3.2, fastapi 0.116.1.
- **Determinism:** a fixed seed (42) for the split, the CV folds and the random forest. The model trained on macOS (Python 3.13) and the one trained inside the Docker image (Python 3.11) give the same probability for the sample patient (0.8254 low risk).
- **Inference script:** scripts/predict_sample.py loads the saved pipeline and scores a JSON patient record outside the API.

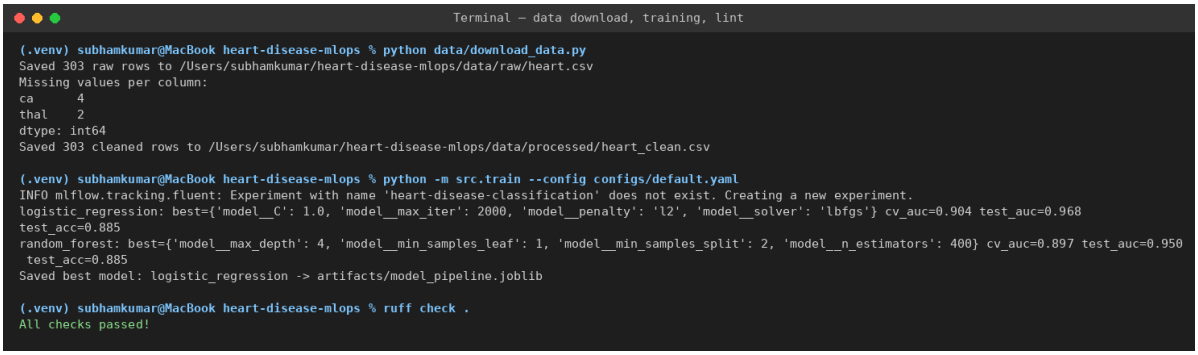


Figure 11. Clean-setup run on macOS: dataset download, training with tuning, and lint.

7. CI/CD pipeline and automated testing

The test suite has 11 Pytest tests:

- tests/test_preprocess.py: transformer layout, dataset loading, imputation and one-hot expansion, and the missing-target error.

- tests/test_model.py: metric ranges, plus an end-to-end training run on synthetic data that must produce a saved pipeline.
- tests/test_api.py: /health, /predict response schema with probabilities summing to 1, and HTTP 422 for invalid input.

The GitHub Actions workflow (.github/workflows/ci.yml) runs on every push and pull request to main, and can also be started manually:

Job	Steps	Failure behaviour
build-test-train	Checkout - Python 3.11 + pip cache - install requirements - ruff check - download data - train - pytest (JUnit XML) - upload artifacts (model, metrics, mlruns, junit.xml)	Any lint error, training exception or failing test exits non-zero and stops the run. Artifacts are uploaded even on failure for debugging.
docker-build-and-smoke-test (needs job 1)	docker build - run container - poll /health - POST sample to /predict - push image to ghcr.io on main	Does not start unless job 1 passes. A failed health check or prediction fails the job.

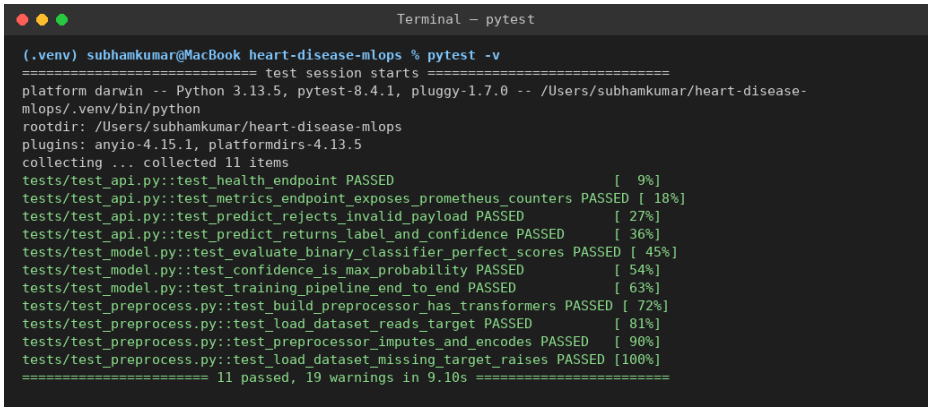


Figure 12. Local Pytest run: 11 tests passed.

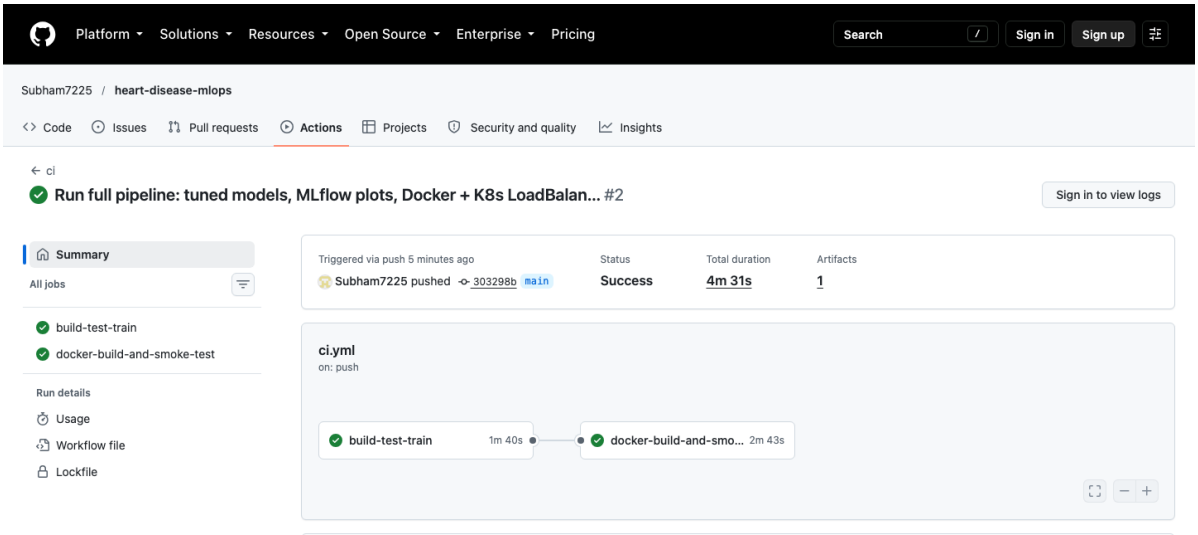


Figure 13. GitHub Actions run for the final commit: both jobs passed (4 min 31 s) and one training-artifact bundle is attached.

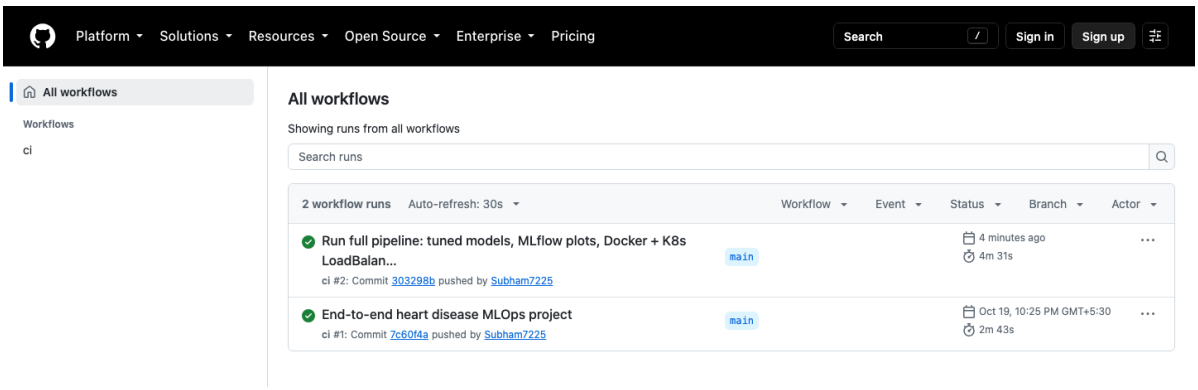


Figure 14. GitHub Actions workflow history.

8. Model containerisation

The Dockerfile uses python:3.11-slim, installs the pinned requirements, then runs the download and training scripts at build time, so the image contains the exact model it serves. It exposes port 8000, defines a curl-based HEALTHCHECK and starts Uvicorn. The FastAPI app validates input with Pydantic (all 13 fields required and type-checked) and returns the predicted class, a label, a confidence and both class probabilities.

```
$ curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" \
  -d '{"age":67,"sex":1,"cp":4,"trestbps":160,"chol":286,"fbs":0,"restecg":2,
       "thalach":108,"exang":1,"oldpeak":1.5,"slope":2,"ca":3,"thal":3}'
{"prediction":1,"label":"high_risk","confidence":0.9833,
 "probabilities":{"low_risk":0.0167,"high_risk":0.9833}}
```

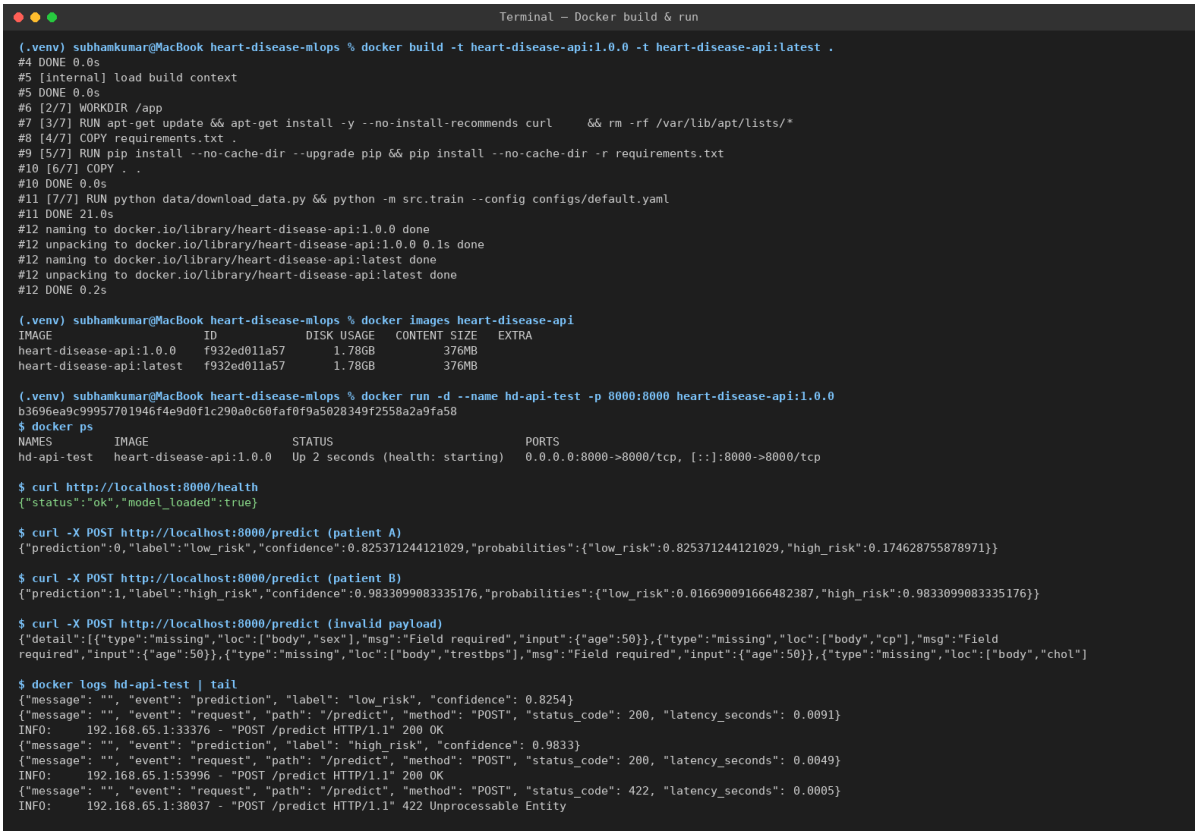


Figure 15. Docker build and container run: /health, two predictions, a rejected invalid payload, and structured logs.

9. Production deployment on Kubernetes

The target is the local Docker Desktop Kubernetes cluster (context docker-desktop, node desktop-control-plane, v1.36.1). The manifests in deployment/ define:

- namespace.yaml and configmap.yaml: an isolated mlops namespace and runtime configuration (APP_ENV) injected as environment variables.
- deployment.yaml: 2 replicas of heart-disease-api:1.0.0, readiness and liveness probes on /health, CPU and memory requests and limits.
- service.yaml: type LoadBalancer on port 8080, forwarding to container port 8000. Docker Desktop publishes it on localhost, and the same manifest provisions a cloud load balancer on GKE, EKS or AKS.
- ingress.yaml: an optional NGINX Ingress for host heart.local. deploy_local.sh applies it only when an NGINX ingress class exists.

deploy_local.sh builds the image, applies the manifests, waits for the rollout, and then calls /health and /predict through the LoadBalancer. An earlier attempt failed with ErrImageNeverPull: Docker Desktop's kind-based cluster cannot see host images when imagePullPolicy is Never. Switching to a versioned tag with imagePullPolicy IfNotPresent lets the cluster pull the locally built image through Docker Desktop's registry mirror.

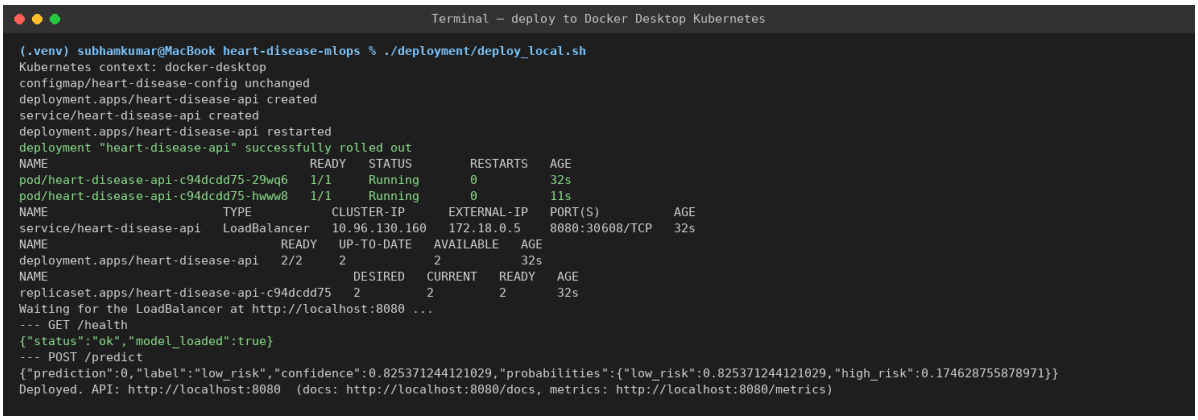


Figure 16. deploy_local.sh: resources created, rollout complete, and endpoint checks through the LoadBalancer.

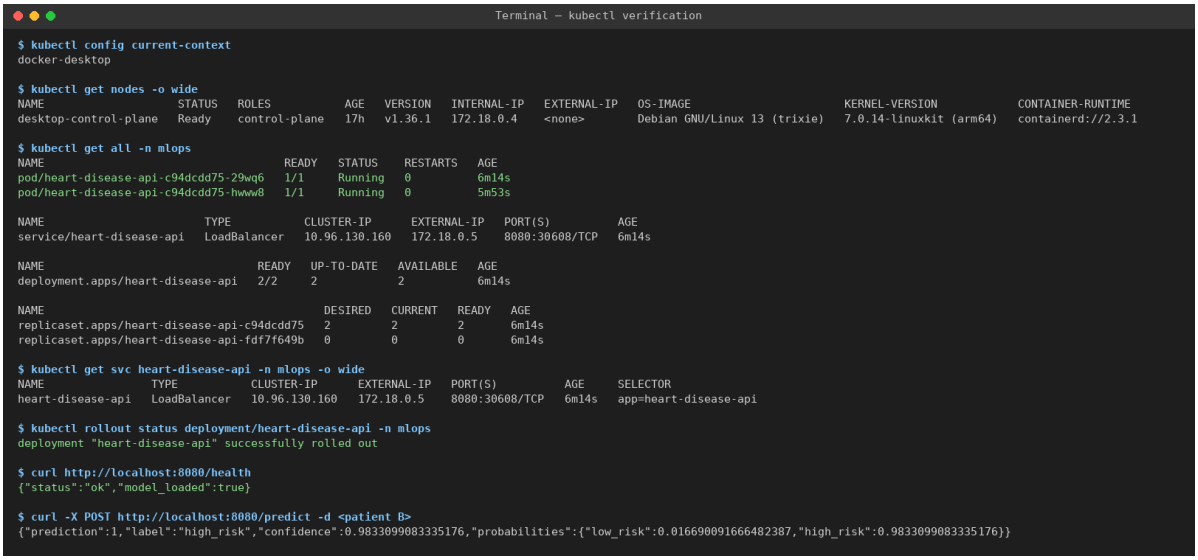


Figure 17. kubectl verification: Ready node, 2/2 pods running, LoadBalancer service, and live /health and /predict responses.

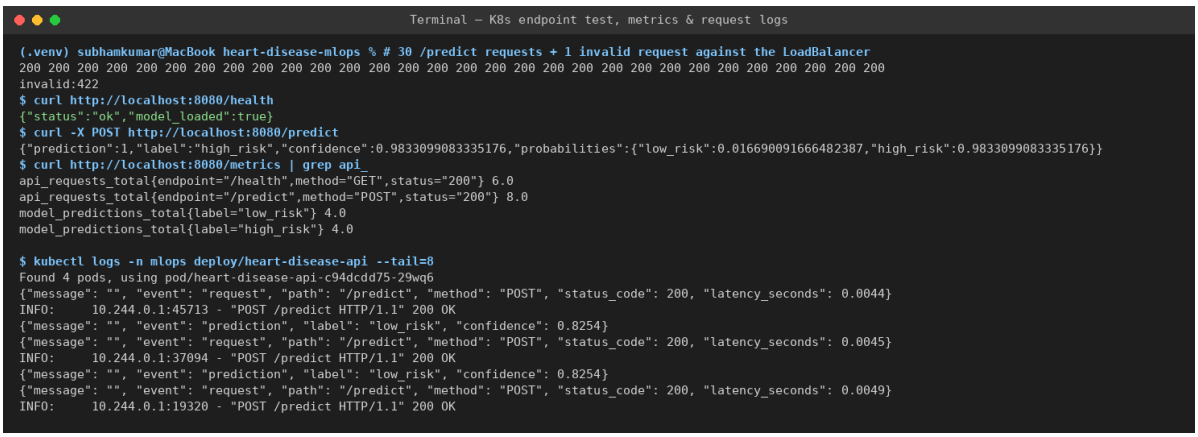


Figure 18. Test traffic against the cluster (30 x HTTP 200, 1 x 422), Prometheus counters and JSON request logs from the pods.

10. Monitoring and logging

- **Request logging:** an HTTP middleware writes one JSON log line per request (path, method, status code, latency). /predict also logs the predicted label and confidence. Logs go to stdout, so kubectl logs and docker logs collect them with no extra agent.
- **Metrics:** /metrics exposes api_requests_total{endpoint, method, status}, the api_request_latency_seconds histogram and model_predictions_total{label}, alongside the standard process metrics.
- **Dashboard:** monitoring/docker-compose.yml starts the API, Prometheus (5 s scrape) and Grafana. The datasource and an 8-panel dashboard are provisioned from files: total requests, predictions served, error responses, p95 latency, request rate by endpoint, mean latency, predictions by class, and responses by status.



Figure 19. Provisioned Grafana dashboard under generated load (451 requests, 252 predictions, 17 rejected inputs, p95 latency 18.6 ms).

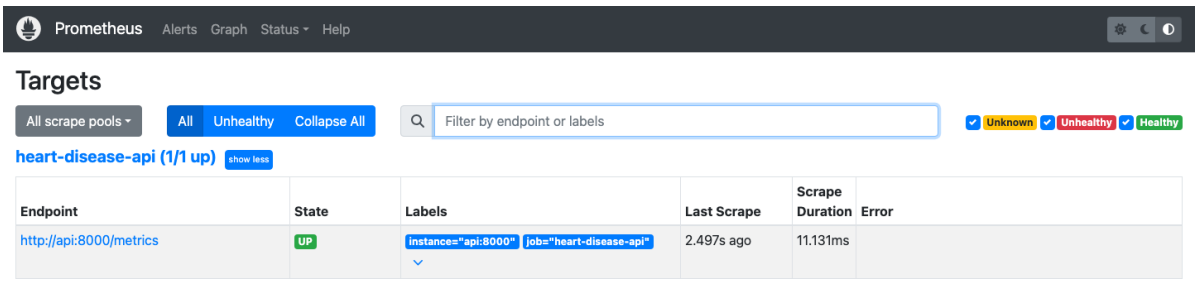


Figure 20. Prometheus target page: the API scrape target is UP.

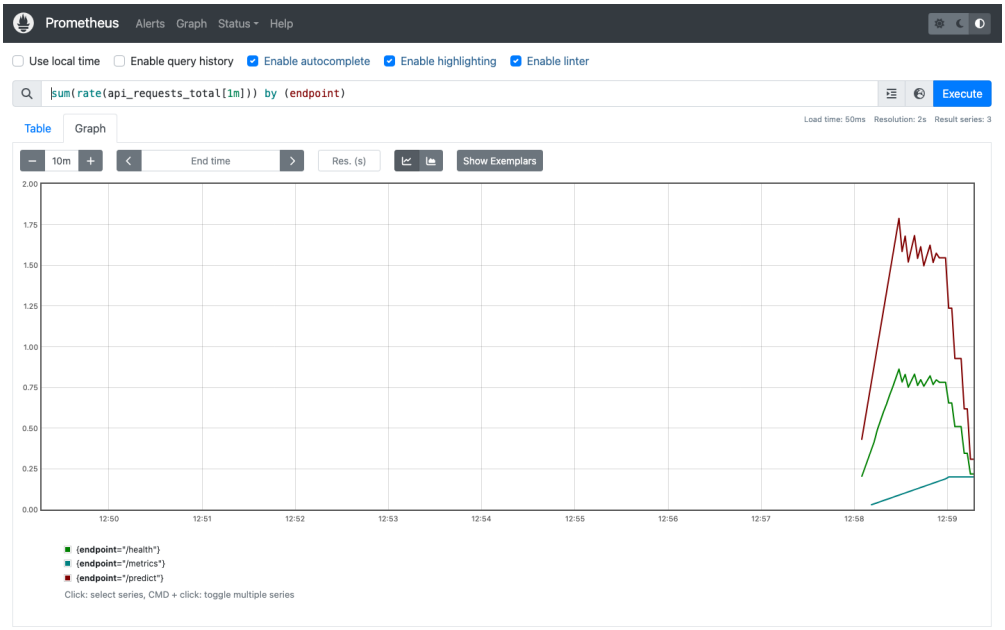


Figure 21. Prometheus query: request rate by endpoint during the load test.

11. Access instructions and deliverables

The API is deployed on local Kubernetes rather than a public cloud, so it is tested as follows after cloning the repository:

```
./deployment/deploy_local.sh              # needs Docker Desktop with Kubernetes on
curl http://localhost:8080/health         # {"status":"ok","model_loaded":true}
open http://localhost:8080/docs           # interactive Swagger UI
curl -X POST http://localhost:8080/predict -H "Content-Type: application/json" \
  -d '{"age":63,"sex":1,"cp":1,"trestbps":145,"chol":233,"fbs":1,"restecg":2,
       "thalach":150,"exang":0,"oldpeak":2.3,"slope":3,"ca":0,"thal":6}'
# fallback if LoadBalancer is unavailable:
kubectl port-forward -n mlops svc/heart-disease-api 8080:8080
```

Deliverable	Location
Code, Dockerfile, requirements.txt	repository root, app/, src/
Cleaned dataset + download script	data/processed/heart_clean.csv, data/download_data.py
Notebooks / scripts (EDA, training, inference)	notebooks/01_eda.ipynb, src/train.py, scripts/predict_sample.py
Unit tests	tests/ (11 tests)
CI workflow	.github/workflows/ci.yml
Deployment manifests	deployment/*.yaml, deployment/deploy_local.sh
Monitoring	monitoring/ (Prometheus, Grafana provisioning, dashboard JSON)

Deliverable	Location
Screenshots	screenshots/
Report / video	reports/final_report.docx, reports/video_walkthrough_script.md

11.1 Limitations and next steps

- The dataset is small (303 patients from one hospital), so the hold-out metrics have wide confidence intervals and the model is not clinically validated.
- Next steps: deploy the GHCR image to a managed cluster (GKE or AKS), add a Horizontal Pod Autoscaler on CPU or request rate, add data-drift checks on incoming features, and register models in the MLflow Model Registry with staged promotion.

Repository: https://github.com/Subham7225/heart-disease-mlops. Dataset: UCI Machine Learning Repository - Heart Disease.